DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA06 – Design and Analysis of Algorithms

CO1 – Assessment Tool 1
IT Application-Based Problem Solving

Scalability Audit of an E-Commerce Platform

Case study: "ShopSphere", an Online Shopping Platform

Student Name	**Dinesh R**
Student Reg. No.	**192372209**
Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual

Parts answered: (a) Understand the Problem · (b) Iterative Modules · (c) Substitution Method · (d) Master Theorem · (e) Best / Average / Worst Cases · (f) Asymptotic Proof · (g) Scalability Decision

Student Declaration and Code of Conduct

I **Dinesh R** (Reg. No. **192372209**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____

Submission particulars

Table 1. Particulars of this submission.

Item	Detail
Student name	Dinesh R
Register number	192372209
Course	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1 (IT Application-Based Problem Solving)
Case study	Scalability audit of the ShopSphere e-commerce platform
Execution-speed assumption	10 basic operations per second (as prescribed in the question paper)
Catalogue sizes audited	n = 100,000 (today) and n = 1,000,000 (within a year)

Answer map

The question paper has seven parts, (a) to (g). The table below shows where each part is answered and which technique it relies on, so that every requirement of the question can be located quickly.

Table 2. Mapping of question parts to the sections of this report.

Part	Question asks for	Module(s)	Main technique
(a)	Input size, basic operation, expected growth	M1 – M5	Operation-count reasoning
(b)	Exact counts, time and space, claim check, n = 1,024 check	M1, M2	Summation of loops
(c)	Backward substitution, closed form, iterative vs recursive	M3 (S2)	Substitution method
(d)	Master Theorem for both designs; extended case	M4, M5	Master Theorem, recursion tree
(e)	Best, average, worst cases; average of S1 with probability p	M3 (S1, S2)	Case analysis, expectation
(f)	Formal proof that f(n) is $\Theta(n^2)$ and not O(n)	Report generator	Definitions of Θ and O
(g)	Operation counts, running times, redesign decisions	M1, M2, S2, M4, M5	Capacity estimate at n = 10^6

Contents

Student Declaration and Code of Conduct . . . 2

Submission particulars . . . 2

Answer map . . . 2

1. Audit Overview and Assumptions . . . 5

1.1 Purpose of the audit . . . 5

1.2 Given data and working assumptions . . . 5

1.3 Method followed for every module . . . 5

1.4 Summary of principal findings . . . 6

2. Mathematical Toolkit . . . 7

2.1 Asymptotic notation . . . 7

2.2 Order of growth . . . 7

2.3 Identities used in the derivations . . . 8

2.4 Master Theorem . . . 8

2.5 Backward substitution . . . 8

Part (a) – Understand the Problem . . . 9

(a.1) Module M1 – Duplicate-Listing Detector . . . 9

(a.2) Module M2 – API Rate Limiter . . . 9

(a.3) Module M3 – Product Search (versions S1 and S2) . . . 10

(a.4) Module M4 – Recommendation Engine . . . 10

(a.5) Module M5 – Sales Analytics Engine and the report generator . . . 10

(a.6) Side-by-side overview . . . 11

Part (b) – Analyse the Iterative Modules . . . 12

(b.1) Module M1 – exact number of comparisons . . . 12

(b.2) Module M2 – exact number of Audit operations . . . 13

Part (c) – Solve a Recurrence by Substitution . . . 17

(c.1) Backward substitution, step by step . . . 17

(c.2) Verification of the closed form . . . 17

(c.3) Asymptotic result and cross-check . . . 18

(c.4) Iterative and recursive implementations . . . 18

Part (d) – Apply the Master Theorem 20

(d.1) Module M4 – current design 20

(d.2) Module M4 – proposed redesign 21

(d.3) What happens when the data doubles 22

(d.4) Module M5 – T(n) = 2T(n/2) + n log n 22

Part (e) – Analyse Best, Average and Worst Cases 25

(e.1) Version S1 – sequential search of the unsorted catalogue 25

(e.2) Version S2 – binary search of the sorted index 26

(e.3) Comparison of S1 and S2 27

Part (f) – Prove an Asymptotic Bound 28

(f.1) What has to be shown 28

(f.2) Lower bound 28

(f.3) Upper bound 28

(f.4) A tighter pair of constants (optional refinement) 28

(f.5) Proof that f(n) is not O(n) 29

(f.6) Where the cost comes from 30

Part (g) – Scalability Decision 31

(g.1) Estimates for n = 1,000,000 31

(g.2) Which modules fail the 10-minute window 32

(g.3) Redesign recommendations 32

(g.4) Decision matrix 33

(g.5) Supplementary: does faster hardware solve the problem? 34

(g.6) Recommended action plan 34

Conclusion 35

Appendix A – Formula Sheet 36

Appendix B – Computational Verification 37

Appendix C – Extended Growth Tables 39

Appendix D – Notation and References 40

Notation 40

References 40

1. Audit Overview and Assumptions

1.1 Purpose of the audit

ShopSphere's product catalogue holds n = 100,000 products today and is expected to grow tenfold to n = 1,000,000 within a year. Before this expansion the CTO needs to know which of five back-end modules will keep working and which will collapse. This report audits the modules M1 to M5 using the tools of algorithm analysis: identifying the basic operation, counting it exactly, solving the resulting sums and recurrences, and stating the result in asymptotic notation. Each result is then interpreted for the platform and used for a redesign decision.

1.2 Given data and working assumptions

Table 3. Parameters and assumptions used throughout the report.

Quantity	Value used	Source / remark
Current catalogue size n_1	100,000	Given in the scenario
Expected catalogue size n_2	1,000,000	Given in the scenario
Server speed	10 basic operations per second	Instruction in the question paper
Nightly processing window	10 minutes = 600 s	Part (g)
Operations that fit in the window	600 s × 10 ops/s = 6,000	Derived; every module is compared with this budget
Logarithm base	2, unless stated otherwise	Halving and doubling give base 2
Calendar conversions	1 day = 86,400 s; 1 year = 365.25 days	Used only to make large times readable
Recurrence base case	T(1) = 1 (constant work on one item)	Standard assumption where none is given
Sizes for recurrences	n a power of 2 for exact solutions	Extended to all n by monotonicity

The prescribed server speed of 10 basic operations per second is very low compared with a real server. It has been used exactly as stated so that the results match the question paper. A supplementary table in Part (g) shows how the conclusions change on faster hardware, because that comparison shows clearly why the growth rate matters more than the machine.

1.3 Method followed for every module

1. **Identify** the input size and the basic operation, the step that is executed most often and dominates the running time.
2. **Count** how many times that operation executes, as an exact function of n, by summing loop bounds or by solving a recurrence.
3. **Simplify** the count and state it in asymptotic notation (Θ where the bound is tight).
4. **Evaluate** the count for n = 100,000 and n = 1,000,000 and convert it to time using 10 operations per second.
5. **Interpret** the figure for the platform and decide whether the module must be redesigned.

1.4 Summary of principal findings

The table below collects the final results derived in Parts (a) to (g). All figures are explained and proved in the sections that follow.

Table 4. Summary of results (details in Parts (b) to (g)).

Module	Exact count (basic operation)	Class	Operations at $n = 10^6$	Time at 10 ops/s
M1 duplicate detector	$n(n-1)/2$ comparisons	$\Theta(n^2)$	5.00×10^{11}	1,584 years
M2 rate limiter	$2^{\lceil \log n \rceil} - 1$ audits	$\Theta(n)$	1,048,575	29.1 hours
M3 search S2	$\lfloor \log n \rfloor + 1$ comparisons	$\Theta(\log n)$	20	2.0 s
M4 current	$2n^2 - n$	$\Theta(n^2)$	2.00×10^{12}	6,338 years
M4 redesign	$n \log n + n$	$\Theta(n \log n)$	2.09×10^{7}	24.2 days
M5 analytics	$n + \frac{1}{2} n \log n (\log n + 1)$	$\Theta(n \log^2 n)$	2.10×10^{8}	242.6 days
Report generator f(n)	$3n^2 + 10n \log n + 50$	$\Theta(n^2)$	3.00×10^{12}	9,507 years

Headline conclusion

With the prescribed 10 operations per second, only the binary search S2 fits in the 10-minute window (6,000 operations). M1 and the current M4 are quadratic and are the most urgent redesign targets. M5 must stop re-sorting at every level. M2 is already linear, so its problem is capacity rather than algorithm design.

2. Mathematical Toolkit

This section collects, in one place, the definitions and identities used later, so that each derivation can refer back to them instead of repeating them.

2.1 Asymptotic notation

Let f and g be functions from the positive integers to the non-negative reals. The three notations used in this report are defined as follows.

Table 5. Formal definitions of O, Ω and Θ.

Notation	Formal definition	Reading
f(n) = O(g(n))	$\exists\, c > 0$, n_0 such that $f(n) \le c \cdot g(n)$ for all $n \ge n_0$	g is an asymptotic upper bound
f(n) = Ω(g(n))	$\exists\, c > 0$, n_0 such that $f(n) \ge c \cdot g(n)$ for all $n \ge n_0$	g is an asymptotic lower bound
f(n) = Θ(g(n))	$\exists\, c_1, c_2 > 0$, n_0 such that $c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for all $n \ge n_0$	g is a tight bound; equivalent to O and Ω together

A bound is **tight** only when it is a Θ bound. Saying that an algorithm is O(n log n) is true for any algorithm that is actually Θ(n), but it hides the fact that the algorithm is much faster. This distinction is the heart of the M2 question in Part (b).

2.2 Order of growth

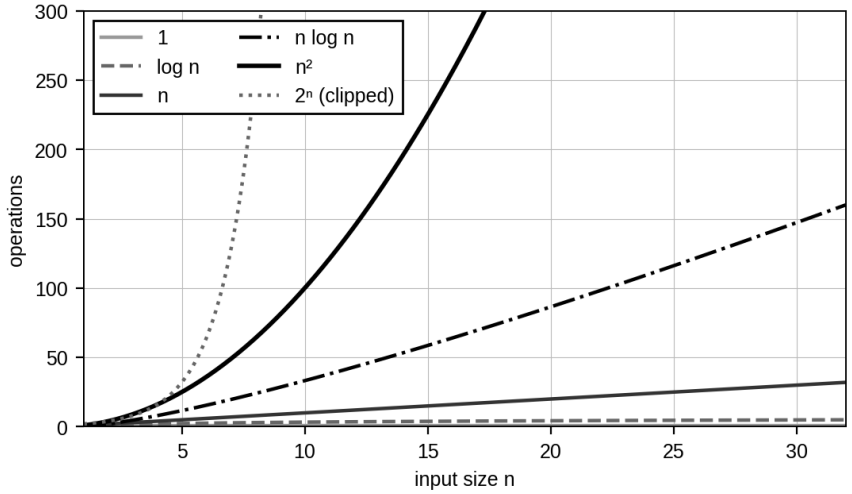


Figure 1. *Common growth rates for small n. The order $1 < \log n < n < n \log n < n^2 < 2^n$ is used to rank the modules.*

For large n the ranking in Figure 1 never changes: any polynomial of higher degree eventually exceeds one of lower degree, and n log n lies strictly between n and n^2. The module analysis in this report places every module somewhere on this ladder.

2.3 Identities used in the derivations

Table 6. Summation and logarithm identities.

Identity	Where it is used
$\Sigma_{k=1}^{m} k = m(m+1)/2$	M1 comparison count; M5 level sums
$\Sigma_{t=0}^{K-1} 2^t = 2^K - 1$	M2 geometric sum of doubling windows
$2^{\log_2 n} = n$ and $\log_2 2^k = k$	All recurrence solutions with $n = 2^k$
$2^{K-1} < n \le 2^K$ where $K = \lceil \log_2 n \rceil$	M2 bounds for non-powers of two
$\log_2 n < n$ for all $n \ge 1$	Upper bound in the proof of Part (f)
$\Sigma_{j=1}^{k} j \cdot 2^{j-1} = (k-1)\cdot 2^k + 1$	Binary search average, Part (e)

2.4 Master Theorem

For a recurrence $T(n) = a \cdot T(n/b) + f(n)$ with $a \ge 1$ and $b > 1$, compare $f(n)$ with the *watershed function* $n^{\log_b a}$:

Table 7. Master Theorem, including the extended form of Case 2.

Case	Condition on f(n)	Result
1	$f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$ (f grows polynomially slower)	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ for some $k \ge 0$ (extended form; $k = 0$ is the basic form)	$T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$
3	$f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$ and $a \cdot f(n/b) \le c \cdot f(n)$ for some $c < 1$	$T(n) = \Theta(f(n))$

2.5 Backward substitution

Backward substitution expands a recurrence repeatedly, replaces the smaller term by its own definition, and looks for a pattern after i expansions. The pattern is then written in terms of i, the expansion is stopped when the argument reaches the base case, and the result is confirmed by induction. It is used in Part (c) for S2 and as a cross-check of the exact solutions in Part (d).

Part (a) – Understand the Problem

For each module this part names the **input size**, the **basic operation** that dominates the running time, the place in the code where the operation occurs, and the growth that should be expected. The detailed counts are derived in Parts (b) to (e). Here the aim is to understand the structure of each module before any formula is written.

(a.1) Module M1 – Duplicate-Listing Detector

Table 8. M1 – problem understanding.

Aspect	Answer
Input size	n = number of products in the catalogue (the length of the array Product).
Basic operation	The comparison of one pair of products: the test *Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller* in the IF statement.
Where it occurs	The innermost statement of the two nested FOR loops. It runs once for every pair (i, j) with i < j.
Expected growth	Quadratic. Each product is compared with every later product, so the number of tests is the number of unordered pairs, which grows like $n^2/2$.

Justification. The IF test is evaluated for every pair whatever the data contain, so it is executed the same number of times in the best, average and worst cases. MarkDuplicate(j) is not a good choice for the basic operation: it runs only when a duplicate is found, so its count depends on the data and it is never executed on a clean catalogue. Each test costs at most two field comparisons, which is a constant, so counting tests is enough to determine the order of growth.

Growth expectation. If n becomes ten times larger, the number of pairs becomes about a hundred times larger. Going from 100,000 to 1,000,000 products therefore multiplies the nightly work by roughly 100.

(a.2) Module M2 – API Rate Limiter

Table 9. M2 – problem understanding.

Aspect	Answer
Input size	n = the bound of the loop, i.e. the number of request records that the windows may cover.
Basic operation	The call Audit(Request[j]), which is marked in the code as the basic operation.
Where it occurs	The inner FOR loop. In one pass of the WHILE loop it runs i times, where i is the current window size.
Expected growth	Linear. The outer loop runs about log n times, but the window size doubles each time, so the total is a geometric series that is dominated by its last term, which is less than n.

Justification. The team member multiplied the number of outer iterations (log n) by the largest inner count (n). That product is only valid if every inner loop ran n times. In this routine the inner loop runs 1, 2, 4, 8, … times, so only the last few iterations are expensive. The earlier iterations together add up to about the same as the last one. This is why the expected growth is linear and not n log n; the claim is examined exactly in Part (b).

(a.3) Module M3 – Product Search (versions S1 and S2)

Table 10. M3 – problem understanding for both versions.

Aspect	S1 – sequential scan	S2 – binary search
Input size	n = number of products in the unsorted catalogue	n = number of entries in the sorted product index
Basic operation	Comparison of the search key with Product[i]	Comparison of the key with the middle entry of the current range
Where it occurs	Loop body; once per element examined	Once per halving step; the range shrinks from n to n/2 each time
Expected growth	Linear: up to n comparisons	Logarithmic: about $\log_2 n$ comparisons

Justification. In both versions the key comparison is the only operation whose count depends on n; the loop bookkeeping happens at most once per comparison. S1 can only discard one element per comparison, but S2 discards half of the remaining entries. When n grows from 100,000 to 1,000,000, S1 may need ten times as many comparisons, whereas S2 needs only about 3.3 more ($\log_2 10 \approx 3.32$).

(a.4) Module M4 – Recommendation Engine

Table 11. M4 – problem understanding.

Aspect	Answer
Input size	n = number of customer–product records handled by one call of the engine.
Basic operation	The combine (merge) step that handles one record while the partial results are joined. It costs a constant per record, so a merge of n records costs n; this is the f(n) = n term in the recurrence.
Where it occurs	At every node of the recursion tree, after the sub-calls return. In the current design each node makes four sub-calls of size n/2; in the redesign it makes two.
Expected growth	Current: the number of sub-problems quadruples while their size only halves, so the work per level doubles and the deepest levels dominate, giving roughly n^2. Redesign: the work per level stays at n for about log n levels, giving roughly n log n.

Justification. The replicated partitions are the reason for the extra sub-problems. Splitting a problem into four halves means that the total size of the sub-problems is $4 \times (n/2) = 2n$, twice the size of the original. Doubling the total size at every level is what makes the growth quadratic. Removing replication restores the sum $2 \times (n/2) = n$ at every level.

(a.5) Module M5 – Sales Analytics Engine and the report generator

Table 12. M5 – problem understanding.

Aspect	Answer
Input size	n = number of sales records.
Basic operation (engine)	The key comparison made while sorting the records by revenue inside one recursion node. A node that holds m records performs about m log m of them; this is f(n) = n log n in the recurrence.
Basic operation (report)	The elementary operation counted by $f(n) = 3n^2 + 10n \log n + 50$. The term $3n^2$ comes from a doubly nested pass over the data, and it dominates the other two terms.
Where it occurs	Engine: in the sort that is repeated at every level of the recursion. Report: in the nightly report generator, mostly in its quadratic part.
Expected growth	Engine: between n log n and n^2. Level i of the recursion tree costs $n(\log n - i)$, and there are log n levels, so the total grows like $n \log^2 n$. Report: quadratic.

Justification. Sorting at every level wastes work: the data at a lower level is a subset of the data that was already sorted above it. The recursion therefore pays a full n log n at the top, then $2 \times (n/2) \log(n/2)$ at the next level, and so on. Summing these costs is done with a recursion tree in Part (d). For the report generator the question gives the cost function directly, so only the dominant term has to be identified; this is proved formally in Part (f).

(a.6) Side-by-side overview

Table 13. Summary of Part (a).

Module	Input size n	Basic operation	Expected growth	Reason
M1	products	pair comparison in IF	Quadratic	every pair compared once
M2	requests	Audit(Request[j])	Linear	doubling windows form a geometric series
M3 – S1	products	key comparison	Linear	one element discarded per comparison
M3 – S2	index entries	key vs middle entry	Logarithmic	half of the range discarded per comparison
M4 current	records	merge of one record	Quadratic	four sub-problems of half size
M4 redesign	records	merge of one record	n log n	two sub-problems of half size
M5	sales records	comparison in per-level sort	n log² n	n(log n − i) per level
Report	data items	operation counted by f(n)	Quadratic	3n² dominates

Table 14. Predicted effect of a tenfold increase in catalogue size (exact counts derived in later parts).

Module	Count at n = 10^5	Count at n = 10^6	Growth factor for n × 10
M1	5.00×10^9	5.00×10^{11}	× 100.00
M2	131,071	1,048,575	× 8.00
M3 – S1 (worst)	100,000	1,000,000	× 10.00
M3 – S2 (worst)	17	20	× 1.18
M4 current	2.00×10^{10}	2.00×10^{12}	× 100.00
M4 redesign	1,760,964	20,931,569	× 11.89
M5	14,724,490	209,599,497	× 14.23
Report f(n)	3.00×10^{10}	3.00×10^{12}	× 99.95

What this means for ShopSphere

A tenfold increase in the catalogue multiplies the work of M1, the current M4 and the report generator by about 100. These three modules scale worst. M5 and the redesigned M4 grow by only 14× and 12×, while S2 grows by a mere 20 ÷ 17, about 1.18×. M2 grows by exactly 8× because of its power-of-two staircase.

Part (b) – Analyse the Iterative Modules

(b.1) Module M1 – exact number of comparisons

The pseudocode of M1 contains two nested loops. The outer loop variable i takes the values 0, 1, …, n − 2, and for each i the inner variable j takes the values i + 1, …, n − 1. The basic operation is the IF test on the pair (i, j).

Step 1 – count the inner loop for a fixed i

For a fixed i the inner loop runs from j = i + 1 to j = n − 1. The number of values is (n − 1) − (i + 1) + 1 = n − 1 − i.

Step 2 – sum over all values of i

$$C_1(n) = \Sigma_{i=0}^{n-2} \Sigma_{j=i+1}^{n-1} 1 \qquad (1)$$

$C_1(n) = \Sigma_{i=0}^{n-2} (n - 1 - i)$	*inner sum of Step 1*
$= (n - 1) + (n - 2) + \ldots + 2 + 1$	*terms for i = 0, 1, …, n − 2*
$= \Sigma_{k=1}^{n-1} k$	*rename k = n − 1 − i*
$= (n - 1)\cdot n / 2$	*sum of the first n − 1 integers*
$= n^2/2 - n/2$	*expanded form*

Result

Exact number of comparisons: $C_1(n) = n(n - 1)/2 = n^2/2 - n/2$.

Step 3 – cross-check by counting pairs

Every unordered pair of distinct products is compared exactly once, with the earlier index in the first position. The number of unordered pairs from n products is the binomial coefficient C(n, 2) = n(n − 1)/2, which agrees with the sum. The table below confirms the formula for small values.

Table 15. M1 count for small n: the loop count and the formula agree.

n	2	3	4	5	6	7	8	10
Pairs checked by loop	1	3	6	10	15	21	28	45
n(n − 1)/2	1	3	6	10	15	21	28	45

Step 4 – asymptotic bounds

For n ≥ 2 we have n − 1 ≥ n/2, so $C_1(n) = n(n - 1)/2 \geq n^2/4$. Also $C_1(n) \leq n^2/2$. With $c_1 = 1/4$, $c_2 = 1/2$ and $n_0 = 2$ the definition of Θ is satisfied, so $C_1(n) \in \Theta(n^2)$.

Step 5 – best, average and worst case

The IF test is evaluated for every pair regardless of the data, so the count is the same in all three cases. The title comparison is made exactly $C_1(n)$ times; because AND is short-circuited, the seller comparison is made only when the titles match and therefore at most $C_1(n)$ times. MarkDuplicate adds at most one more constant-time call per pair. The total running time is therefore $\Theta(n^2)$ in the best, average and worst cases.

Step 6 – space complexity

The routine uses the two loop counters i and j and nothing else that depends on n. The auxiliary space is Θ(1), that is, constant. (The catalogue itself occupies Θ(n) memory, but it is the input and is not counted as extra space.)

Table 16. M1 numerical results.

Measure	n = 100,000	n = 1,000,000
Comparisons n(n − 1)/2	4,999,950,000	499,999,500,000
Time at 10 ops/s	16 years	1,584 years
Time complexity	Θ(n²)	Θ(n²)
Auxiliary space	Θ(1)	Θ(1)

What this means for ShopSphere

The nightly duplicate scan needs about 5×10^9 comparisons today and about 5×10^{11} after the expansion, which is 100 times more. At the prescribed 10 operations per second this is about 1,584 years. The routine uses almost no memory, so memory is not the issue; the quadratic time is. M1 must be redesigned (Part (g)).

(b.2) Module M2 – exact number of Audit operations

The WHILE loop doubles i after every pass, starting from i = 1, and continues while i < n. In each pass the inner loop calls Audit for j = 0, …, i − 1, which is i calls.

Step 1 – values taken by i

Number the passes t = 0, 1, 2, …. In pass t the value of i is 2^t. The loop body is executed as long as $2^t < n$, that is, as long as $t < \log_2 n$. The largest such integer t is $\lceil \log_2 n \rceil - 1$, so the number of passes is $K = \lceil \log_2 n \rceil$ (for $n \geq 2$).

Step 2 – Audit calls in each pass

In pass t the inner loop runs $i = 2^t$ times, so it makes 2^t Audit calls.

Step 3 – total as a geometric series

$$A(n) = \sum_{t=0}^{K-1} 2^t \qquad \text{one term per pass of the WHILE loop}$$

$$= 1 + 2 + 4 + \ldots + 2^{K-1} \qquad \text{write out the terms}$$

$$= 2^K - 1 \qquad \text{geometric series with ratio 2}$$

$$= 2^{\lceil \log_2 n \rceil} - 1 \qquad \text{substitute } K = \lceil \log_2 n \rceil$$

Result

Exact number of Audit operations: $A(n) = 2^{\lceil \log_2 n \rceil} - 1$. For n a power of two this is exactly $n - 1$.

Step 4 – bounds for every n

Because $K = \lceil \log_2 n \rceil$, we have $2^{K-1} < n \leq 2^K$. The right-hand inequality gives $A(n) = 2^K - 1 \geq n - 1$. The left-hand inequality, being between integers, gives $2^{K-1} \leq n - 1$, hence $2^K \leq 2n - 2$ and $A(n) \leq 2n - 3$. Therefore

$$n - 1 \leq A(n) \leq 2n - 3 \text{ for all } n \geq 2$$

For n ≥ 2 we have n − 1 ≥ n/2, so $n/2 \le A(n) \le 2n$, and $A(n) \in \Theta(n)$ with $c_1 = 1/2$, $c_2 = 2$, $n_0 = 2$.

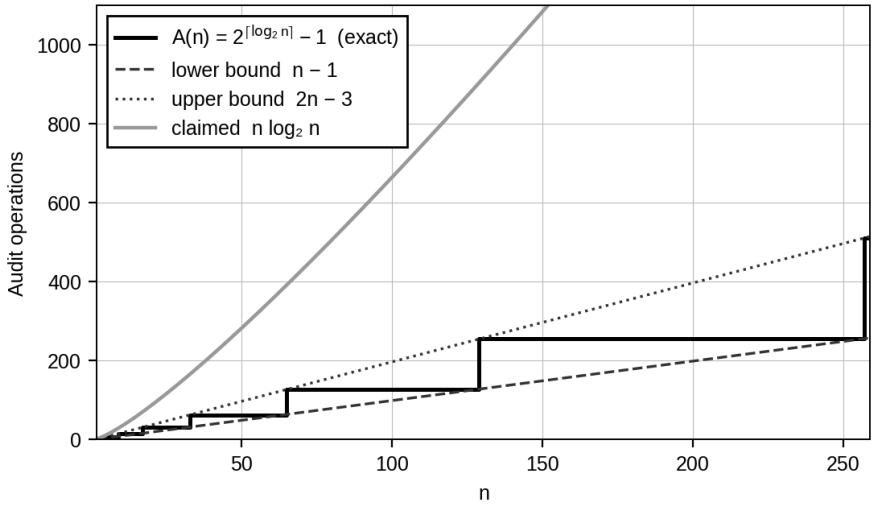


Figure 2. *Exact Audit count A(n) (staircase) lies between n − 1 and 2n − 3. The curve n log n that the team member claimed grows far above it.*

Step 5 – is the claim Θ(n log n) correct?

No. The claim is not correct as a tight bound. The reasoning "the outer loop runs about log n times and the inner loop runs up to n times" is a valid way to get an *upper bound*, and indeed $A(n) \le 2n - 3 \le n \log n$ for n ≥ 4, so A(n) = O(n log n) is a true statement. But a Θ bound also needs a matching lower bound, and that fails:

Suppose $A(n) \ge c \cdot n \cdot \log_2 n$ for all $n \ge n_0$	*assumption for contradiction (c > 0)*
Then $2n - 3 \ge A(n) \ge c \cdot n \cdot \log_2 n$	*upper bound from Step 4*
so $\log_2 n \le 2/c$ for all $n \ge n_0$	*divide by c·n*
But $\log_2 n$ is unbounded as n grows	*contradiction*

The same fact follows from the limit $\lim A(n)/(n \log n) = 0$. Hence A(n) is *not* Ω(n log n), and the claim Θ(n log n) is wrong. The inner loop does not run n times in each of the log n passes; it runs 1, 2, 4, … times, so the sum is geometric and is dominated by the last pass. The correct tight bound is

Result

Tight bound for M2: Θ(n), not Θ(n log n). Time complexity Θ(n) in the best, average and worst cases (the loop never depends on data); auxiliary space Θ(1).

Step 6 – verification for n = 1,024

Here n = 1,024 = 2^{10}, so K = ⌈$\log_2$ 1024⌉ = 10 passes are expected, and A(n) = 2^{10} − 1 = 1,023. The trace below runs the loop by hand. The loop stops when i reaches 1,024 because the condition 1,024 < 1,024 is false.

Table 17. Hand trace of M2 for n = 1,024.

Pass t	$i = 2^t$	Is i < 1,024?	Audit calls in pass	Running total
0	1	yes	1	1
1	2	yes	2	3
2	4	yes	4	7
3	8	yes	8	15
4	16	yes	16	31
5	32	yes	32	63
6	64	yes	64	127
7	128	yes	128	255
8	256	yes	256	511
9	512	yes	512	1,023
10	1,024	no – loop ends	0	1,023

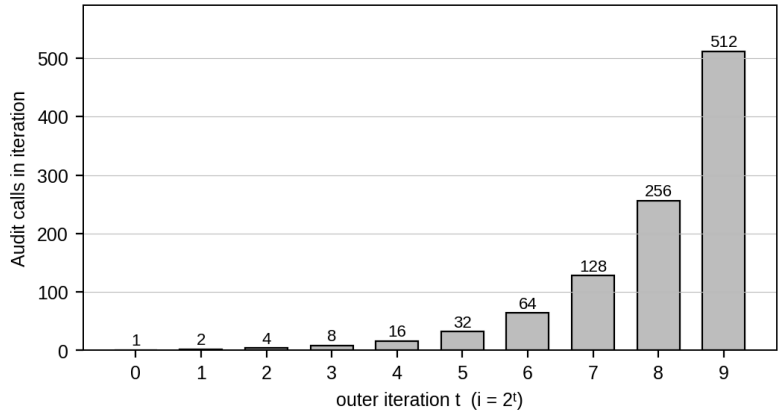


Figure 3. *Audit calls per pass for n = 1,024: the last pass alone (512) is about half of the total of 1,023.*

Verification result

Passes: $10 = \log_2 1{,}024$ (matches). Total Audit calls: $1 + 2 + \ldots + 512 = \mathbf{1{,}023} = 2^{10} - 1 = n - 1$ (matches the formula).

The team member's formula would predict $n \log n = 1{,}024 \times 10 = 10{,}240$ operations, which is about 10 times too large.

The formula is also sensitive to the position of n between powers of two. For n = 1,025 the loop runs one more pass with i = 1,024, so $A(1{,}025) = 2^{11} - 1 = 2{,}047 = 2n - 3$, exactly the upper bound of Step 4. The table below shows several sizes.

Table 18. M2 exact count versus the claimed n log n.

n	$K = \lceil \log_2 n \rceil$	$A(n) = 2^K - 1$	A(n) / n	$n \log_2 n$ (claim)	Claim ÷ exact
8	3	7	0.875	24	3.4
100	7	127	1.270	664	5.2
1,000	10	1,023	1.023	9,966	9.7
1,024	10	1,023	0.999	10,240	10.0
1,025	11	2,047	1.997	10,251	5.0
100,000	17	131,071	1.311	1,660,964	12.7
1,000,000	20	1,048,575	1.049	19,931,569	19.0

Table 19. M2 numerical results.

Measure	n = 100,000	n = 1,000,000
Passes K	17	20
Audit operations A(n)	131,071	1,048,575
Time at 10 ops/s	3.6 hours	29.1 hours
Time / space complexity	Θ(n) / Θ(1)	Θ(n) / Θ(1)

What this means for ShopSphere

M2 is already a linear-time routine, so the team member's fear of an n log n cost is unfounded: at n = 1,000,000 the exact cost is 1,048,575 audits, about 19 times less than the 19,931,569 predicted by the claim. The work grows by exactly 8× (from 131,071 to 1,048,575) when the catalogue grows by 10×, because the staircase moves from the 2^{17} to the 2^{20} step. M2 does not need an algorithmic redesign; it needs capacity (Part (g)).

Part (c) – Solve a Recurrence by Substitution

Search version S2 works on a sorted product index. It compares the key with the middle entry and discards half of the index after every comparison. The number of comparisons satisfies

$$T(n) = T(n/2) + 1,\ T(1) = 1 \qquad (2)$$

The term T(n/2) is the cost of searching the remaining half; the term 1 is the single comparison with the middle entry. T(1) = 1 means that a range with one entry needs one comparison. To solve the recurrence exactly, take $n = 2^k$, so that n/2 is always an integer; the general case is treated at the end.

(c.1) Backward substitution, step by step

Start from the recurrence and replace the smaller term by its own definition, again and again.

$T(n) = T(n/2) + 1$	*original recurrence (expansion 0)*
$= [\ T(n/4) + 1\ ] + 1 = T(n/4) + 2$	*use T(n/2) = T(n/4) + 1*
$= [\ T(n/8) + 1\ ] + 2 = T(n/8) + 3$	*use T(n/4) = T(n/8) + 1*
$= [\ T(n/16) + 1\ ] + 3 = T(n/16) + 4$	*use T(n/8) = T(n/16) + 1*
…	
$= T(n/2^i) + i$	*pattern after i expansions*

Proof that the pattern holds for every i

For i = 1 the pattern is the recurrence itself. Assume $T(n) = T(n/2^i) + i$. The recurrence applied to the argument $n/2^i$ gives $T(n/2^i) = T(n/2^{i+1}) + 1$. Substituting this, $T(n) = T(n/2^{i+1}) + 1 + i = T(n/2^{i+1}) + (i + 1)$, which is the pattern for i + 1. By induction it holds for all $i \geq 1$.

Reaching the base case

The expansion stops when the argument becomes 1, that is when $n/2^i = 1$. With $n = 2^k$ this gives $i = k = \log_2 n$. Then

$T(n) = T(n/2^k) + k$	*pattern with i = k*
$= T(1) + \log_2 n$	*since $n/2^k = 1$ and $k = \log_2 n$*
$= 1 + \log_2 n$	*base case T(1) = 1*

Result

Exact closed form: $T(n) = \log_2 n + 1$ for $n = 2^k$; and for every $n \geq 1$, $T(n) = \lfloor \log_2 n \rfloor + 1$.

(c.2) Verification of the closed form

By induction (guess-and-verify)

Base: n = 1 gives $T(1) = 1$ and $\log_2 1 + 1 = 0 + 1 = 1$. **Step:** assume $T(n/2) = \log_2(n/2) + 1 = \log_2 n$. Then $T(n) = T(n/2) + 1 = \log_2 n + 1$. So the closed form is correct for all powers of two.

Extension to every n

When n is not a power of two the range is halved with a floor, giving $T(n) = T(\lfloor n/2 \rfloor) + 1$. Let $2^k \le n < 2^{k+1}$, i.e. $k = \lfloor \log_2 n \rfloor$. Then $2^{k-1} \le \lfloor n/2 \rfloor < 2^k$, so $\lfloor n/2 \rfloor$ has floor-logarithm $k - 1$. By induction on k, $T(\lfloor n/2 \rfloor) = (k - 1) + 1 = k$ and therefore $T(n) = k + 1 = \lfloor \log_2 n \rfloor + 1$.

By direct computation

Table 20. Recurrence values and the closed form agree.

n	1	2	4	8	16	1,024	100,000	1,000,000
T(n) from the recurrence	1	2	3	4	5	11	17	20
$\lfloor \log_2 n \rfloor + 1$	1	2	3	4	5	11	17	20

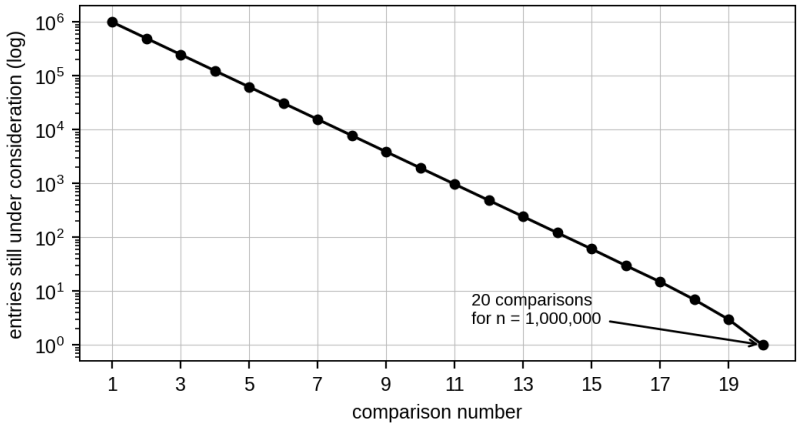


Figure 4. *Entries still under consideration in S2 for n = 1,000,000: the range halves 19 times and the 20th comparison examines the last entry.*

The halving sequence for n = 1,000,000 is 1,000,000 → 500,000 → 250,000 → 125,000 → 62,500 → 31,250 → 15,625 → 7,812 → 3,906 → 1,953 → 976 → 488 → 244 → 122 → 61 → 30 → 15 → 7 → 3 → 1. It has 20 terms, which matches $\lfloor \log_2 1{,}000{,}000 \rfloor + 1 = 19 + 1 = 20$.

(c.3) Asymptotic result and cross-check

For $n \ge 2$, $\log_2 n \le \log_2 n + 1 \le 2 \log_2 n$, so with $c_1 = 1$, $c_2 = 2$, $n_0 = 2$ we get $T(n) \in \Theta(\log n)$.

Master Theorem cross-check. Here a = 1, b = 2, f(n) = 1. The watershed function is $n^{\log_2 1} = n^0 = 1$. Since $f(n) = \Theta(1) = \Theta(n^0 \cdot \log^0 n)$, Case 2 with k = 0 applies and gives $T(n) = \Theta(n^0 \cdot \log n) = \Theta(\log n)$, which agrees with the substitution result.

(c.4) Iterative and recursive implementations

Both implementations perform the same comparisons, so the time complexity is the same. They differ in how they use memory.

```
BinarySearchIterative(A[0..n-1], key)
  lo <- 0 ; hi <- n - 1
  WHILE lo <= hi DO
    mid <- floor((lo + hi) / 2)
    IF A[mid] = key THEN RETURN mid
    ELSE IF key < A[mid] THEN hi <- mid - 1
    ELSE lo <- mid + 1
  RETURN -1
```

Pseudocode of the iterative form of S2.

```
BinarySearchRecursive(A, lo, hi, key)
  IF lo > hi THEN RETURN -1
  mid <- floor((lo + hi) / 2)
  IF A[mid] = key THEN RETURN mid
  IF key < A[mid] THEN RETURN BinarySearchRecursive(A, lo, mid - 1, key)
  ELSE RETURN BinarySearchRecursive(A, mid + 1, hi, key)
```

Pseudocode of the recursive form of S2.

Table 21. Comparison of the two implementations of S2.

Property	Iterative S2	Recursive S2
Comparisons (worst case)	$\lfloor \log_2 n \rfloor + 1$	$\lfloor \log_2 n \rfloor + 1$
Time complexity	Θ(log n) worst and average; Θ(1) best	Θ(log n) worst and average; Θ(1) best
Auxiliary space	**Θ(1)**: variables lo, hi, mid, key only	**Θ(log n)**: one stack frame per call, depth $\lfloor \log_2 n \rfloor + 1$
Stack frames at n = 1,000,000	none beyond the caller	20 frames
Recurrence for the space	S(n) = O(1)	S(n) = S(n/2) + O(1) ⇒ Θ(log n)
Remark	Preferred in production	Clear, but the call is a tail call; a compiler with tail-call optimisation would reduce it to Θ(1)

The recursive version stores, for every active call, its parameters lo, hi and key and a return address, which is a constant amount per frame. The recursion depth equals the number of halvings, $\lfloor \log_2 n \rfloor + 1$, so the stack grows logarithmically. For n = 1,000,000 that is only 20 frames, which is negligible, but the iterative version needs no stack at all.

Result for Part (c)

$T(n) = \lfloor \log_2 n \rfloor + 1$, hence time Θ(log n). Space: iterative Θ(1); recursive Θ(log n).

What this means for ShopSphere

S2 needs at most 20 comparisons for the whole catalogue of one million products, which takes 2 seconds at 10 operations per second. It grows by only three comparisons when the catalogue grows tenfold. The price is that the index must be kept sorted; the one-time sorting or batch maintenance cost is outside this recurrence. For search, S2 is the version to keep.

Part (d) – Apply the Master Theorem

(d.1) Module M4 – current design

The recommendation engine splits the data into four parts of size n/2 and merges in linear time:

$$T(n) = 4T(n/2) + n \tag{3}$$

Step 1 – identify the parameters

Table 22. Master Theorem parameters for T(n) = 4T(n/2) + n.

Symbol	Meaning	Value for the current design
a	number of sub-problems	4
b	factor by which the size shrinks	2
f(n)	cost of dividing and merging	n
$\log_b a$	exponent of the watershed function	$\log_2 4 = 2$
$n^{\log_b a}$	watershed function	n^2

Step 2 – compare f(n) with the watershed function

The merge cost f(n) = n is polynomially smaller than n^2. Since $\log_2 4 = 2$, choosing $\varepsilon = 1$ gives $f(n) = n = O(n^{2-\varepsilon})$, which is exactly the condition of **Case 1**.

Step 3 – conclude

Result
Case 1 applies, so $T(n) = \Theta(n^{\log_2 4}) = \Theta(n^2)$ for the current design.

Step 4 – cross-check with the recursion tree

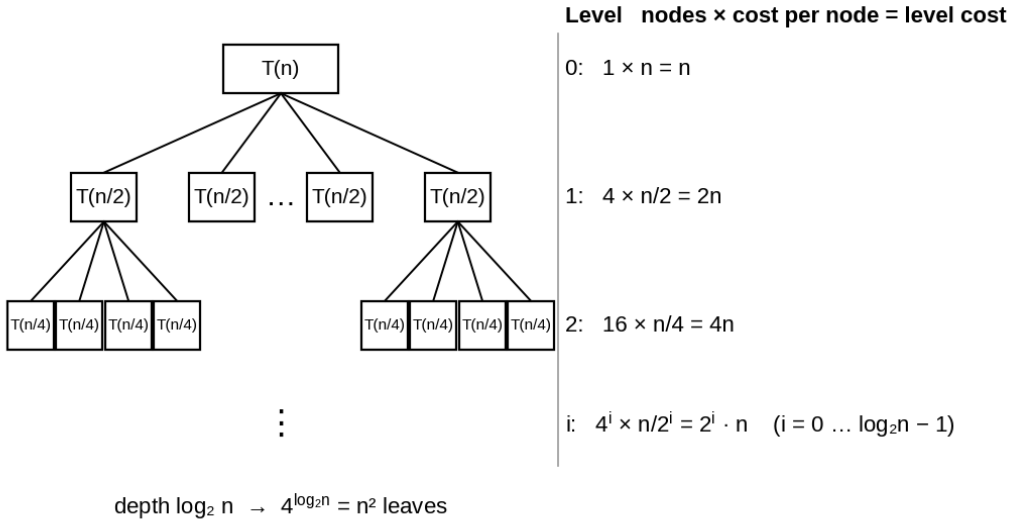


Figure 5. *Recursion tree of the current M4: the work per level doubles, so the leaves dominate.*

At level i there are 4^i nodes, each merging $n/2^i$ records, so the level costs $4^i \cdot n/2^i = 2^i \cdot n$. Summing over the $\log_2 n$ merge levels i = 0, …, $\log_2 n - 1$ gives $n(2^{\log_2 n} - 1) = n(n - 1)$. The $4^{\log_2 n} = n^2$ leaves cost 1 each.

Hence

$$T(n) = n(n-1) + n^2 \qquad \textit{merge levels + leaves}$$

$$= 2n^2 - n \qquad \textit{exact solution } (T(1) = 1)$$

Check: T(2) = 4·T(1) + 2 = 6 and $2\cdot2^2 - 2 = 6$; T(4) = 4·6 + 4 = 28 and $2\cdot4^2 - 4 = 28$. The exact solution is $\Theta(n^2)$, in agreement with Case 1.

(d.2) Module M4 – proposed redesign

Removing the replication leaves only two sub-problems of half size:

$$T(n) = 2T(n/2) + n \tag{4}$$

Table 23. Master Theorem parameters for T(n) = 2T(n/2) + n.

Symbol	Meaning	Value for the redesign
a	number of sub-problems	2
b	factor by which the size shrinks	2
f(n)	cost of dividing and merging	n
$\log_b a$	exponent of the watershed function	$\log_2 2 = 1$
$n^{\log_b a}$	watershed function	n

Now $f(n) = n = \Theta(n^1) = \Theta(n^{\log_2 2})$, so f(n) has exactly the same growth as the watershed function: **Case 2** with k = 0.

Result
Case 2 applies, so $T(n) = \Theta(n^{\log_2 2}\cdot\log n) = \Theta(n \log n)$ for the redesign.

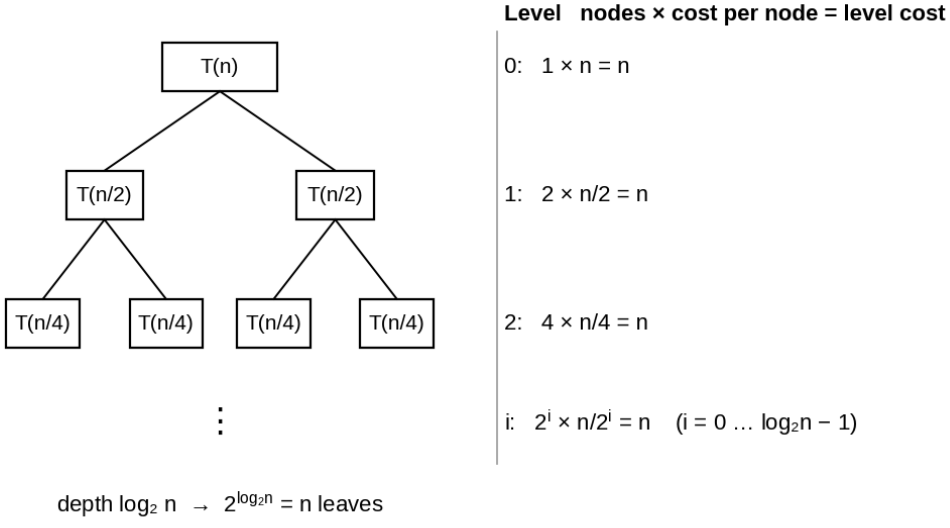


Figure 6. *Recursion tree of the redesigned M4: every level costs exactly n.*

Each of the $\log_2$ n levels costs $2^i\cdot n/2^i = n$, and the n leaves cost 1 each, so $T(n) = n\cdot\log_2 n + n$. Check: T(2) = 2·1 + 2 = 4 = 2·1 + 2; T(4) = 2·4 + 4 = 12 = 4·2 + 4.

(d.3) What happens when the data doubles

Replace n by 2n in the exact solutions.

$$T_{\text{cur}}(2n) / T_{\text{cur}}(n) = (8n^2 - 2n) / (2n^2 - n) \qquad \textit{from } T_{cur}(n) = 2n^2 - n$$

$$= (8n - 2) / (2n - 1) = 4 + 2/(2n - 1) \qquad \textit{simplify; tends to 4}$$

$$T_{\text{new}}(2n) / T_{\text{new}}(n) = (2n \cdot \log_2 n + 4n) / (n \cdot \log_2 n + n) \qquad \textit{from } T_{new}(n) = n \log_2 n + n$$

$$= 2 + 2/(\log_2 n + 1) \qquad \textit{simplify; tends to 2 from above}$$

Table 24. Effect of growth in the data on both designs of M4.

Catalogue grows	Current M4 ratio	Redesign ratio	Meaning
n → 2n (doubling)	× 4.000 (quadruples)	× 2.096 (just over double)	doubling data quadruples the old design but only slightly more than doubles the new one
$10^5 \rightarrow 10^6$ (× 10)	× 100.00	× 11.89	a tenfold catalogue costs 100× versus 11.9×

At n = 1,000,000 the redesign needs 20,931,569 operations instead of 1,999,999,000,000, an improvement of about 95,549 times.

> **What this means for ShopSphere**
>
> The current engine is a quadratic algorithm: every time the product and customer data double, the nightly work quadruples, and ten times more data means a hundred times more work. The redesign behaves almost linearly; doubling the data costs about 2.1 times more. Removing the replicated partitions is therefore the single most valuable change for the recommendation engine.

(d.4) Module M5 – T(n) = 2T(n/2) + n log n

Step 1 – parameters

Here a = 2, b = 2 and $f(n) = n \log_2 n$. The watershed function is $n^{\log_2 2} = n$.

Step 2 – test the three standard cases

Table 25. The three standard cases of the Master Theorem fail for M5.

Case	Needed	What happens here	Applies?
1	$f(n) = O(n^{1-\varepsilon})$ for some ε > 0	n log n grows *faster* than n, so it cannot be $O(n^{1-\varepsilon})$	No
2 (basic, k = 0)	$f(n) = \Theta(n)$	n log n is larger than n by a factor log n that is unbounded	No
3	$f(n) = \Omega(n^{1+\varepsilon})$ for some ε > 0	$f(n)/n = \log n$, but $\log n = o(n^{\varepsilon})$ for every ε > 0, so f is not polynomially larger	No

To see that Case 3 fails, note that $\lim \log n / n^{\varepsilon} = 0$ for every ε > 0 (for instance by L'Hôpital's rule), so no polynomial gap of size n^{ε} exists between f(n) and the watershed function. The recurrence falls in the gap between Cases 2 and 3, so the three standard cases do **not** apply.

Step 3 – extended Master Theorem (Case 2 with k = 1)

In the extended form, if $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ with k ≥ 0, then $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$. Here $f(n) = n \log n = \Theta(n^1 \cdot \log^1 n)$, so k = 1.

Result
Extended Case 2 with k = 1 gives $T(n) = \Theta(n \cdot \log^2 n)$ for M5.

Step 4 – derivation by recursion tree

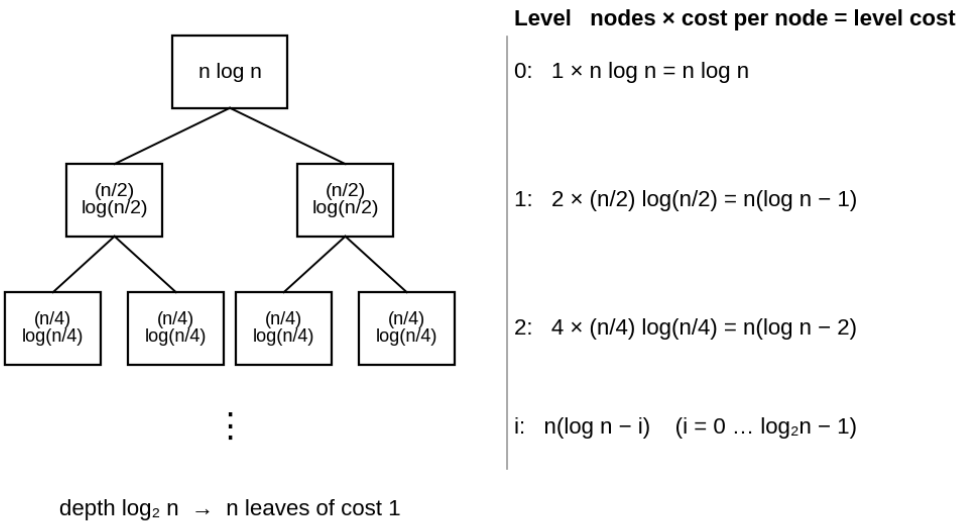


Figure 7. *Recursion tree of M5: level i costs n(log n − i).*

At level i there are 2^i nodes, each sorting $n/2^i$ records at cost $(n/2^i)\cdot\log_2(n/2^i) = (n/2^i)(k - i)$ where k = $\log_2$ n. A level therefore costs $n(k - i)$. There are k levels (i = 0, …, k − 1), plus n leaves of cost 1.

$$T(n) = n + \Sigma_{i=0}^{k-1}\, n(k - i) \qquad \textit{leaves + level costs}$$
$$= n + n\cdot(k + (k - 1) + \ldots + 1) \qquad \textit{write the sum out}$$
$$= n + n\cdot k(k + 1)/2 \qquad \textit{sum of the first k integers}$$
$$= n + \tfrac{1}{2}\, n \log_2^2 n + \tfrac{1}{2}\, n \log_2 n \qquad \textit{substitute } k = \log_2 n$$

Verification by induction. Assume $T(n/2) = n/2 + (n/2)(k - 1)k/2$. Then $2T(n/2) + n\cdot k = n + n(k - 1)k/2 +$ nk = $n + n\cdot k(k + 1)/2$. The base case T(1) = 1 matches k = 0.

Step 5 – asymptotic constants

For n ≥ 4 (k ≥ 2) we have $(1/2)\cdot n\cdot k^2 \le T(n) \le n\cdot k^2$: the lower bound is one term of the sum, and the upper bound holds because $n + nk/2 \le nk^2/2$ is equivalent to $(k - 2)(k + 1) \ge 0$. Hence $T(n) \in \Theta(n \log^2 n)$ with $c_1 = 1/2$, $c_2 = 1$, $n_0 = 4$.

Step 6 – independent confirmation (Akra–Bazzi)

With p = 1 the Akra–Bazzi solution is $T(n) = \Theta(n(1 + \int_1^n f(u)/u^2\, du))$. Here $f(u)/u^2 = \log u / u$, whose integral from 1 to n is proportional to $\log^2 n$. Hence $T(n) = \Theta(n \log^2 n)$ again.

Table 26. M5 exact solution compared with reference orders ($n = 2^k$).

n	Exact T(n)	n log n	n log² n	n²
1,024	57,344	10,240	102,400	1,048,576
32,768	3,964,928	491,520	7,372,800	1,073,741,824
1,048,576	221,249,536	20,971,520	419,430,400	1,099,511,627,776

What this means for ShopSphere

M5 grows faster than n log n but much slower than n^2. The extra log n factor comes entirely from re-sorting the data at every level of the recursion. At n = 1,000,000 the engine needs about 2.1×10^8 operations; sorting once and merging would need only about 2.1×10^7, ten times fewer (Part (g)).

Part (e) – Analyse Best, Average and Worst Cases

This part analyses the two search versions of module M3. In both versions the basic operation is the comparison of the search key with a catalogue entry. The three cases are defined by the input condition that makes the number of comparisons smallest, typical and largest.

(e.1) Version S1 – sequential search of the unsorted catalogue

Table 27. S1 – best, average and worst case.

Case	Input condition	Comparisons	Complexity
Best	The searched product is the **first** entry of the catalogue	1	Θ(1)
Average	The product exists with probability p, and when it exists it is equally likely to be at any of the n positions	n(1 − p/2) + p/2 (derived below)	Θ(n)
Worst	The product is the **last** entry, or is **not in** the catalogue at all	n	Θ(n)

Derivation of the average number of comparisons

Let p be the probability that the searched product exists. Assume that, when it exists, each of the n positions is equally likely, so that position i has probability p/n. The two kinds of outcome are:

- **Successful search** ending at position i (i = 1, …, n): it uses exactly i comparisons and has probability p/n.
- **Unsuccessful search** (product absent): every entry is examined, so it uses n comparisons and has probability 1 − p.

$$C_{\text{avg}}(n, p) = \Sigma_{i=1}^{n}\, i\cdot(p/n) + n\cdot(1 - p) \qquad \textit{expected value over all outcomes}$$

$$= (p/n)\cdot\Sigma_{i=1}^{n}\, i + n(1 - p) \qquad \textit{take the constant p/n out of the sum}$$

$$= (p/n)\cdot n(n + 1)/2 + n(1 - p) \qquad \textit{sum of the first n integers}$$

$$= p(n + 1)/2 + n - \text{np} \qquad \textit{cancel n}$$

$$= n - \text{np}/2 + p/2 \qquad \textit{collect terms}$$

$$= n(1 - p/2) + p/2 \qquad \textit{final form}$$

Result

$C_{\text{avg}}(n, p) = n(1 - p/2) + p/2$. Special cases: p = 1 gives $(n + 1)/2$ (the product is certainly present); p = 0 gives n (it is certainly absent).

Check with a small example

For n = 4 and p = 0.5 the formula gives 4 × 0.75 + 0.25 = 3.25. Directly: positions 1 to 4 each have probability 0.5/4 = 0.125, contributing (1 + 2 + 3 + 4) × 0.125 = 1.25, and the absent case contributes 0.5 × 4 = 2, a total of 3.25.

Evaluation for n = 1,000,000 and p = 0.5

$$C_{\text{avg}} = 1{,}000{,}000 \times (1 - 0.25) + 0.25 \qquad p/2 = 0.25$$
$$= 1{,}000{,}000 \times 0.75 + 0.25 \qquad \text{simplify}$$
$$= 750{,}000.25 \qquad \text{comparisons on average}$$

Result

For n = 1,000,000 and p = 0.5, S1 needs on average **750,000.25 comparisons**, i.e. 75,000 s ≈ 20.8 hours at 10 operations per second.

Table 28. Sensitivity of the S1 average to the hit probability p.

p (probability product exists)	0	0.25	0.5	0.75	1
$C_{\text{avg}}(n = 10^6, p)$	1,000,000	875,000.125	750,000.25	625,000.375	500,000.5
Time at 10 ops/s	27.8 hours	24.3 hours	20.8 hours	17.4 hours	13.9 hours

(e.2) Version S2 – binary search of the sorted index

Table 29. S2 – best, average and worst case.

Case	Input condition	Comparisons	Complexity
Best	The key equals the **middle** entry of the whole index, so the first comparison succeeds	1	Θ(1)
Average	The key is a randomly chosen entry (successful), or a random value that is absent (unsuccessful)	≈ $\log_2 n - 1$ (successful); ≈ $\log_2 n$ (unsuccessful)	Θ(log n)
Worst	The key is **absent**, or sits at the deepest level of the halving process	$\lfloor \log_2 n \rfloor + 1$	Θ(log n)

Worst case

Each comparison halves the range, and the recurrence T(n) = T(n/2) + 1 of Part (c) counts the comparisons in the worst case, giving $\lfloor \log_2 n \rfloor + 1$. For n = 1,000,000 this is 20 comparisons.

Average case (successful search)

To keep the arithmetic exact, take $n = 2^k - 1$, so that the comparisons form a perfectly balanced tree. The middle entry is examined in comparison 1 (1 entry), the two entries in the middle of the halves in comparison 2, and in general 2^{j-1} entries are found in comparison j, for j = 1, …, k. If each entry is equally likely to be the key,

$$C_{\text{succ}} = (1/n)\cdot\Sigma_{j=1}^{k}\, j\cdot 2^{j-1} \qquad j \text{ comparisons for each of } 2^{j-1} \text{ entries}$$
$$= ((k-1)\cdot 2^k + 1) / (2^k - 1) \qquad \text{standard sum } \Sigma\, j\cdot 2^{j-1} = (k-1)2^k + 1$$
$$= (k-1) + k/(2^k - 1) \qquad \text{write } (k-1)2^k + 1 = (k-1)(2^k - 1) + k$$
$$\approx \log_2(n+1) - 1 \qquad \text{since } 2^k = n + 1 \text{ and } k/(2^k - 1) \to 0$$

Check with n = 7 (k = 3): the formula gives 2 + 3/7 = 2.4286; directly, (1·1 + 2·2 + 3·4)/7 = 17/7 = 2.4286.

Average case (unsuccessful search) and the effect of p

In a balanced tree every unsuccessful search ends after about k comparisons, that is, about $\log_2 n$. Combining both outcomes with the existence probability p,

$$C_{\text{avg}}^{S2}(n, p) \approx p\cdot(\log_2 n - 1) + (1 - p)\cdot\log_2 n = \log_2 n - p$$

For n = 1,000,000 and p = 0.5 this is about 19.93 − 0.5 ≈ **19.4 comparisons**, which takes about 1.9 seconds at 10 operations per second.

(e.3) Comparison of S1 and S2

Table 30. Best, average and worst case numbers for both versions.

Case	S1 (n = 10^6)	S2 (n = 10^6)	Ratio S1 ÷ S2
Best	1	1	1
Average, p = 0.5	750,000.25	≈ 19.4	≈ 38,600
Worst	1,000,000	20	50,000

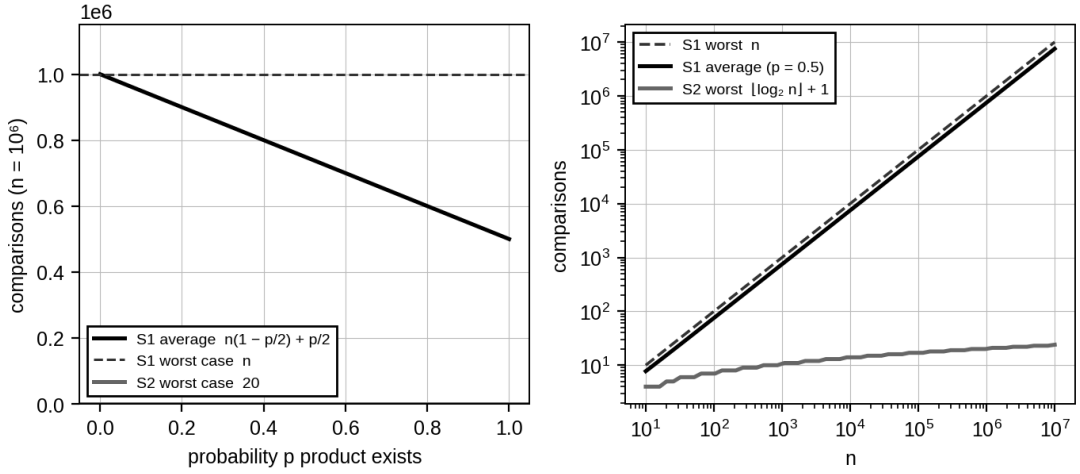


Figure 8. *Left: S1 average cost against p for n = 10^6, compared with the worst case of S2. Right: growth of both versions with n.*

What this means for ShopSphere

Both versions are equally fast when the key happens to be the first entry or the middle entry, but this is rare. In ordinary use S1 examines hundreds of thousands of products per search while S2 examines about twenty. As the catalogue grows tenfold, S1 grows tenfold and S2 grows by three comparisons. A search service that scans linearly will become the bottleneck of the storefront long before the batch jobs do, so S1 should be retired for catalogue lookups.

Part (f) – Prove an Asymptotic Bound

The nightly report generator performs

$$f(n) = 3n^2 + 10n \cdot \log_2 n + 50 \tag{5}$$

operations. The task is to prove, from the formal definitions, that $f(n) \in \Theta(n^2)$ by exhibiting constants c_1, c_2 and n_0, and to show that $f(n) \notin O(n)$.

(f.1) What has to be shown

By the definition of Θ in Section 2.1, we must find constants $c_1 > 0$, $c_2 > 0$ and $n_0 \geq 1$ such that

$$c_1 \cdot n^2 \leq 3n^2 + 10n \cdot \log_2 n + 50 \leq c_2 \cdot n^2 \text{ for all } n \geq n_0$$

(f.2) Lower bound

For every n ≥ 1 we have $\log_2 n \geq 0$, so the two other terms are non-negative: $10n \cdot \log_2 n \geq 0$ and $50 > 0$. Therefore

$$f(n) = 3n^2 + 10n \cdot \log_2 n + 50 \geq 3n^2 \text{ for all } n \geq 1$$

This gives $c_1 = 3$ with $n_0 = 1$.

(f.3) Upper bound

Two simple facts are needed for n ≥ 1.

1. **$\log_2$ n ≤ n.** This holds because $n < 2^n$ for every n ≥ 1 (true at n = 1 as 1 < 2, and doubling the right side when n increases by 1 keeps it larger). Taking logarithms gives $\log_2 n < n$.
2. **$1 \leq n^2$.** Obviously true for n ≥ 1, hence $50 \leq 50n^2$.

Using fact 1 in the middle term and fact 2 in the last term:

$f(n) = 3n^2 + 10n \cdot \log_2 n + 50$	*definition*
$\leq 3n^2 + 10n \cdot n + 50$	*fact 1: $\log_2 n \leq n$*
$\leq 3n^2 + 10n^2 + 50n^2$	*fact 2: $50 \leq 50n^2$*
$= 63n^2$	*add the coefficients*

Result

For all n ≥ 1: $3n^2 \leq f(n) \leq 63n^2$. Hence $\boldsymbol{c_1 = 3, c_2 = 63, n_0 = 1}$ satisfy the definition and $f(n) \in \Theta(n^2)$.

(f.4) A tighter pair of constants (optional refinement)

The constant 63 is generous because the term 50 dominates for small n. A tighter upper constant can be found by asking when $10n \cdot \log_2 n + 50 \leq n^2$, so that $f(n) \leq 4n^2$. The table tests this inequality near its crossing point.

Table 31. Finding n_0 for $c_2 = 4$.

n	10n·log₂ n + 50	n²	Inequality 10n·log₂ n + 50 ≤ n²
10	382.2	100	fails
20	914.4	400	fails
50	2,871.9	2,500	fails
59	3,520.8	3,481	fails
60	3,594.1	3,600	holds
64	3,890.0	4,096	holds
100	6,693.9	10,000	holds
1000	99,707.8	1,000,000	holds

The inequality first holds at n = 60 (3,594.1 ≤ 3,600). It continues to hold for all larger n because $g(n) = n^2 - 10n\cdot\log_2 n - 50$ is increasing there: its derivative $g'(n) = 2n - 10(\log_2 n + 1/\ln 2)$ equals about 46.5 at n = 60 and increases with n. So $c_2 = 4$ and $n_0 = 60$ also work, which shows $3n^2 \le f(n) \le 4n^2$ for all $n \ge 60$.

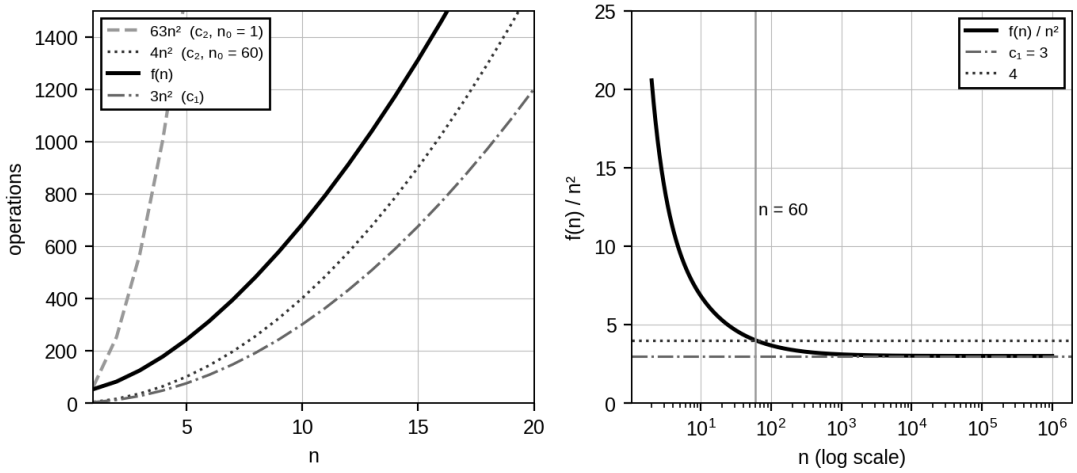


Figure 9. *Left: f(n) lies between 3n² and 63n² from n = 1. Right: the ratio f(n)/n² falls towards 3 and stays between 3 and 4 from n = 60.*

(f.5) Proof that f(n) is not O(n)

Proof by contradiction. Suppose f(n) = O(n). Then there are constants c > 0 and n_0 such that $f(n) \le c\cdot n$ for all $n \ge n_0$. From Section (f.2) we know $f(n) \ge 3n^2$. Combining the two,

$3n^2 \le f(n) \le c\cdot n$ for all $n \ge n_0$	*assumption and lower bound*
$3n \le c$	*divide by n > 0*
$n \le c/3$ for all $n \ge n_0$	*divide by 3*

But the right-hand side is a fixed number, whereas n can be taken as large as we like. Choose $n = \max(n_0, \lceil c/3 \rceil + 1)$. Then n > c/3, which contradicts $n \le c/3$. So no such c and n_0 exist, and f(n) is **not** O(n).

Confirmation by a limit. $\lim_{n\to\infty} f(n)/n = \lim (3n + 10\cdot\log_2 n + 50/n) = \infty$. Since the ratio is unbounded, f grows strictly faster than n. The same argument shows f(n) is not O(n log n) either, because f(n)/(n log n) ≈ 3n/log n → ∞.

(f.6) Where the cost comes from

Table 32. Contribution of each term to f(n).

n	3n²	10n·log₂ n	50	f(n)	Share of 3n²
10	300	332	50	682	43.98 %
100	30,000	6,644	50	36,694	81.76 %
1,000	3,000,000	99,658	50	3,099,708	96.78 %
10,000	300,000,000	1,328,771	50	301,328,821	99.56 %
1,000,000	3,000,000,000,000	199,315,686	50	3,000,199,315,736	99.99 %

Result for Part (f)

$f(n) = 3n^2 + 10n \log n + 50 \in \Theta(n^2)$, with $c_1 = 3$, $c_2 = 63$, $n_0 = 1$ (or $c_2 = 4$, $n_0 = 60$), and $f(n) \notin O(n)$.

What this means for ShopSphere

At n = 1,000,000 the report generator needs f(n) = 3,000,199,315,736 operations, of which 99.99 % come from the 3n² term; the n log n term and the constant are negligible. Tuning the logarithmic part would achieve nothing. The only way to make the report scale is to remove the quadratic behaviour. At 10 operations per second this report would take about 9,507 years.

Part (g) – Scalability Decision

This part converts the operation counts of Parts (b) to (f) into running times for n = 1,000,000 at the prescribed 10 basic operations per second, tests each module against the nightly processing window of 10 minutes, and recommends which modules must be redesigned.

(g.1) Estimates for n = 1,000,000

The window of 10 minutes is 600 seconds. At 10 operations per second a server can perform 600 × 10 = **6,000 operations** in that time. A module fits the window only if its operation count is at most 6,000. The required throughput in the table is the number of operations divided by 600 seconds.

Table 33. Operations and running time at n = 1,000,000 (10 operations per second).

Module	Count formula	Operations	Time at 10 ops/s	Fits 600 s?	Throughput needed (ops/s)
M1 duplicate detector	$n(n-1)/2$	5.00×10^{11}	1,584 years	No	833,332,500
M2 rate limiter	$2^{\lceil \log n \rceil} - 1$	1,048,575	29.1 hours	No	1,748
S2 binary search	$\lfloor \log n \rfloor + 1$	20	2.0 s	**Yes**	0.033
M4 current	$2n^2 - n$	2.00×10^{12}	6,338 years	No	3.33×10^{9}
M4 redesign	$n \log n + n$	20,931,569	24.2 days	No	34,886
M5 analytics	$n + \tfrac{1}{2} n \log n (\log n + 1)$	209,599,497	242.6 days	No	349,332

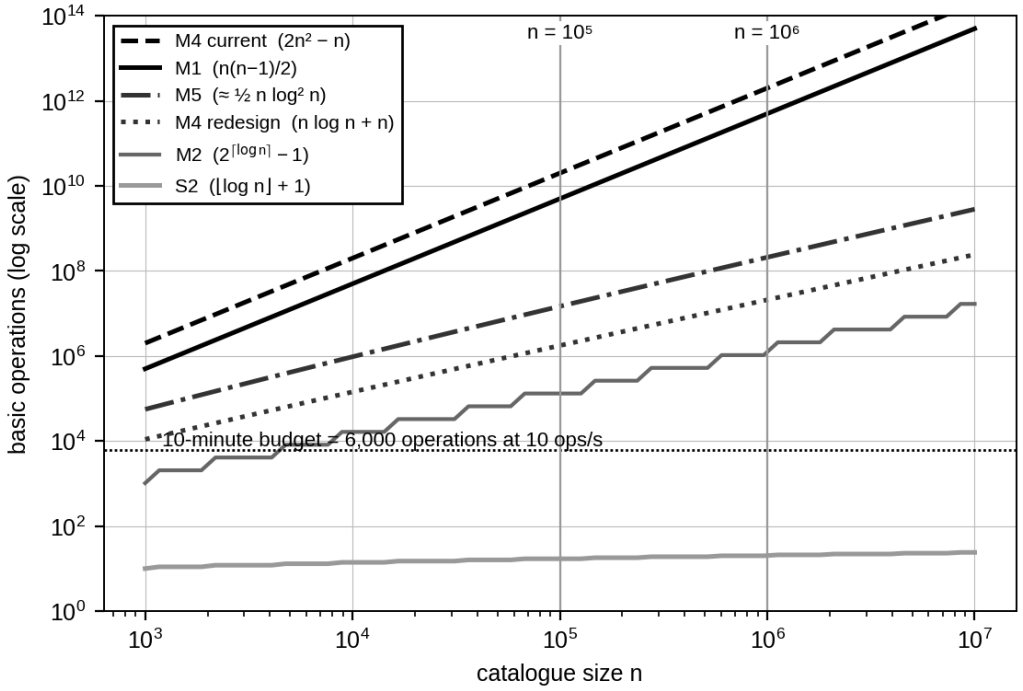


Figure 10. *Operation counts of all modules against n (log–log scale). Only S2 stays below the 6,000-operation budget at n = 10^6.*

(g.2) Which modules fail the 10-minute window

Table 34. Comparison with the 600 s window.

Module	Time at n = 10^6	Times the 600 s window	Verdict
M1 duplicate detector	1,584 years	83,333,250×	**Fails**
M2 rate limiter	29.1 hours	175×	**Fails**
S2 binary search	2.0 s	< 0.01×	Fits
M4 current	6,338 years	333,333,167×	**Fails**
M4 redesign	24.2 days	3,489×	**Fails**
M5 analytics	242.6 days	34,933×	**Fails**

Modules that fail: M1, M2, M4 (both the current design and the redesign) and M5. **Module that passes:** only S2, which needs 20 comparisons or 2 seconds. In order of severity the failures are the current M4 (about 6,300 years), M1 (about 1,600 years), M5 (about 243 days), the redesigned M4 (about 24 days) and M2 (about 29 hours). The report generator of Part (f) is worse than all of them (about 9,500 years) and is also not acceptable.

The failures are of two different kinds, which is important for deciding what to do:

- **Algorithmic failures** – the growth rate itself is too high: M1 (n^2), the current M4 (n^2), the report generator (n^2) and M5 ($n \log^2 n$, avoidable).
- **Capacity failures** – the growth rate is already the best achievable for the task (n log n or n), but the prescribed 10 operations per second is too slow for a million items: the redesigned M4 and M2.

(g.3) Redesign recommendations

M1 – replace the quadratic scan with hashing or sorting (must be redesigned)

The duplicates are defined by equality of (title, seller). Equality can be tested without comparing all pairs, in two ways.

Table 35. Alternative designs for M1.

Design	Idea	Time	Extra space	Operations at n = 10^6
Current nested loops	Compare every pair	$\Theta(n^2)$	$\Theta(1)$	5.00×10^{11}
Sort and scan	Sort by (seller, title), then compare neighbours only	$\Theta(n \log n)$	$\Theta(1)$ to $\Theta(n)$	20,931,568
Hash set (recommended)	Insert key (seller, title) into a hash table; a key already present is a duplicate	$\Theta(n)$ expected	$\Theta(n)$	1,000,000

The hash-set design lowers the count from 5×10^{11} to about 10^6, roughly 500,000 times fewer operations. Even better, the check can be made **when a listing is uploaded**: one hash lookup per new listing, expected $\Theta(1)$, so the nightly job only processes the listings added that day and the quadratic job disappears. The cost of this design is memory for the table, which is $\Theta(n)$ and affordable for a million short keys.

M4 – adopt the redesign and parallelise (must be redesigned)

Removing the replicated partitions changes the recurrence from $4T(n/2) + n$, which is $\Theta(n^2)$, to $2T(n/2) + n$, which is $\Theta(n \log n)$, as shown by Case 1 and Case 2 of the Master Theorem. The operation count drops from 2.00×10^{12} to 2.09×10^{7}, about 95,549 times less. The two sub-problems of every node are independent, so the redesigned engine also distributes naturally across servers. Even so, 20,931,569 operations is not 6,000, so at the prescribed speed it still requires roughly 3,489 servers in parallel (or faster processors) to finish in 10 minutes; recomputing only the partitions that changed since the previous night would reduce this further.

M5 – sort once instead of at every level (must be redesigned)

The recurrence $2T(n/2) + n \log n$ is $\Theta(n \log^2 n)$ because every node re-sorts data that is already sorted at the level below it. If the halves are sorted recursively and only *merged* in linear time, the recurrence becomes $2T(n/2) + n$, which is $\Theta(n \log n)$ by Case 2. At $n = 1{,}000{,}000$ this reduces the count from 2.10×10^{8} to 2.09×10^{7}, a saving of a factor of 10.0. The saving is a factor of about $\frac{1}{2} \cdot \log n$ and therefore grows with n.

M3 – use S2 (or a hash index) for all searches

S1 must be retired for catalogue lookups: it needs 750,000 comparisons on average and 1,000,000 in the worst case at $n = 10^{6}$, against 20 for S2. The sorted index has to be maintained, which can be done by batching new products and merging them in $\Theta(n)$ per batch or by using a balanced tree; a hash index would reduce the expected search to $\Theta(1)$ when only exact-match lookups are needed.

M2 – keep the algorithm, fix the claim, add capacity

M2 is $\Theta(n)$, and every algorithm that audits every request must read each request at least once, so $\Omega(n)$ is also a lower bound; no asymptotic redesign is possible. Auditing only the requests that are new since the previous window could remove at most the repeated audits, which is 48,575 of 1,048,575 calls (about 5 %) at $n = 1{,}000{,}000$. The remaining lever is capacity: audits are independent, so about 175 servers working in parallel would finish within 10 minutes. The document that describes M2 should be corrected from $\Theta(n \log n)$ to $\Theta(n)$ so that capacity planning is not overstated.

Report generator – remove the quadratic term

The $3n^2$ term accounts for 99.99 % of the cost. If it can be replaced by sorting or hash-based aggregation, only the $10n \log n$ term remains, about 1.99×10^{8} operations instead of 3.00×10^{12}, a gain of about 15,052 times.

(g.4) Decision matrix

Table 36. Decision matrix after the recommended redesigns (servers counted at 10 operations per second each).

Module	Decision	New class	Operations at 10^6	Time at 10 ops/s	Servers for 600 s
M1	Redesign (hash set)	$\Theta(n)$ expected	1,000,000	27.8 hours	167
M2	Keep; add capacity	$\Theta(n)$	1,048,575	29.1 hours	175
M3 – S1	Retire; use S2	$\Theta(\log n)$	20	2.0 s	1
M4	Redesign (remove replication)	$\Theta(n \log n)$	20,931,569	24.2 days	3,489
M5	Redesign (merge, not re-sort)	$\Theta(n \log n)$	20,931,569	24.2 days	3,489
Report	Redesign (remove $3n^2$)	$\Theta(n \log n)$	199,315,736	230.7 days	33,220

The last column makes the main message of the audit concrete: with the prescribed hardware, no redesigned module except the search fits one server in 10 minutes, but the redesigns reduce the number of required servers by factors between 10 and 500,000 compared with the original algorithms. Without them, M4 alone would need more than 300 million servers.

(g.5) Supplementary: does faster hardware solve the problem?

This table is not part of the prescribed assumption. It uses a rate of 10^9 operations per second for one server, to show how far hardware alone could go.

Table 37. Running time at 10^9 operations per second.

Module	n = 10^6: time	n = 10^7: time	Verdict at 10^7
M1 current	8.3 min	13.9 hours	**Fails**
M4 current	33.3 min	2.3 days	**Fails**
M4 redesign	0.021 s	0.243 s	Fits 600 s
M5 current	0.210 s	2.8 s	Fits 600 s
M2	0.001 s	0.017 s	Fits 600 s

At this speed M1 would just fit the window at n = 1,000,000 (500 s), but one more tenfold growth to 10,000,000 products multiplies its time by 100 to about 14 hours, whereas the redesigned algorithms stay well within the window. Fast hardware therefore only postpones the problem; the growth rate decides how long the postponement lasts.

(g.6) Recommended action plan

1. **Immediate:** replace S1 by S2 in every search path (one-day change, 50,000-fold fewer comparisons in the worst case).
2. **Priority 1:** adopt the redesigned recommendation engine, $T(n) = 2T(n/2) + n$, and parallelise its two independent sub-problems.
3. **Priority 2:** replace the nested-loop duplicate detector by a hash-set check at upload time.
4. **Priority 3:** change the sales analytics engine to sort once and merge, and remove the quadratic part of the report generator.
5. **Planning:** keep M2 as it is, correct its documented complexity to $\Theta(n)$, and provision parallel capacity for it.

Conclusion

The audit analysed five modules of ShopSphere with the same method: identify the basic operation, count it exactly, express the count asymptotically and evaluate it at the current and the expected catalogue sizes. The findings are as follows.

- M1 performs $n(n-1)/2$ comparisons, so it is $\Theta(n^2)$ in time and $\Theta(1)$ in extra space. It will need 100 times more work after the expansion.
- M2 performs $2^{\lceil \log n \rceil} - 1$ audits and is $\Theta(n)$; the claim of $\Theta(n \log n)$ is only an upper bound and is not tight. For $n = 1{,}024$ the exact count is 1,023.
- Search S2 satisfies $T(n) = T(n/2) + 1$ and is exactly $\lfloor \log_2 n \rfloor + 1$ comparisons, $\Theta(\log n)$; its space is $\Theta(1)$ iteratively and $\Theta(\log n)$ recursively.
- M4 is $\Theta(n^2)$ in its current design (Master Theorem Case 1) and $\Theta(n \log n)$ after the redesign (Case 2). M5 falls in the gap between the standard cases and is $\Theta(n \log^2 n)$.
- S1 needs 750,000.25 comparisons on average for $n = 10^6$ and $p = 0.5$, while S2 needs about 19.4.
- The report generator cost $3n^2 + 10n \log n + 50$ is $\Theta(n^2)$ with constants 3, 63 and $n_0 = 1$, and is not $O(n)$.

The main lesson is that the asymptotic class matters more than the speed of the machine. The three quadratic components (M1, the current M4 and the report generator) are the ones that must be redesigned first. The recommended changes reduce the number of required basic operations by factors from about 10 (M5) to about 500,000 (M1), and they keep the platform workable when the catalogue keeps growing beyond one million products.

Appendix A – Formula Sheet

Table 38. Summary of all derived formulas.

Module / result	Recurrence or count	Exact solution	Class
M1 comparisons	$\Sigma\ (n-1-i)$	$n(n-1)/2$	$\Theta(n^2)$
M2 Audit calls	$\Sigma\ 2^t,\ t < \lceil \log n \rceil$	$2^{\lceil \log_2 n \rceil} - 1$ ($n - 1$ if $n = 2^k$)	$\Theta(n)$
S2 comparisons	$T(n) = T(n/2) + 1,\ T(1) = 1$	$\lfloor \log_2 n \rfloor + 1$	$\Theta(\log n)$
M4 current	$T(n) = 4T(n/2) + n,\ T(1) = 1$	$2n^2 - n$	$\Theta(n^2)$
M4 redesign	$T(n) = 2T(n/2) + n,\ T(1) = 1$	$n \log_2 n + n$	$\Theta(n \log n)$
M5	$T(n) = 2T(n/2) + n \log n,\ T(1) = 1$	$n + \frac{1}{2}\, n \log_2 n\, (\log_2 n + 1)$	$\Theta(n \log^2 n)$
S1 average	$p \cdot \Sigma\ i/n + (1-p) \cdot n$	$n(1 - p/2) + p/2$	$\Theta(n)$
S2 average (successful)	$\Sigma\ j \cdot 2^{j-1} / n$	$(k-1) + k/(2^k - 1) \approx \log_2 n - 1$	$\Theta(\log n)$
Report f(n)	$3n^2 + 10n \log n + 50$	$3n^2 \le f(n) \le 63n^2,\ n \ge 1$	$\Theta(n^2)$

Table 39. Reference values used in the numerical evaluations.

Quantity	n = 100,000	n = 1,000,000
$\log_2 n$	16.6096	19.9316
$\lceil \log_2 n \rceil$	17	20
$\lfloor \log_2 n \rfloor + 1$	17	20
$n \log_2 n$	1,660,964	19,931,569
$n \log_2^2 n$	27,588,016	397,267,426
n^2	10,000,000,000	1,000,000,000,000

Appendix B – Computational Verification

Every closed form in this report was checked by executing the algorithms with operation counters. The Python program below counts the basic operations of M1, M2 and S2 by actually running the loops, evaluates the recurrences of M4 and M5 directly, and tests the bounds used in Part (f). Its output follows the listing.

```
import math, random

def m1_count(n):
    c = 0
    for i in range(0, n - 1):
        for j in range(i + 1, n):
            c += 1
    return c

def m2_count(n):
    i, c, passes = 1, 0, 0
    while i < n:
        for j in range(i):
            c += 1
        i *= 2
        passes += 1
    return c, passes

def bsearch(A, key):
    lo, hi, comps = 0, len(A) - 1, 0
    while lo <= hi:
        mid = (lo + hi) // 2
        comps += 1
        if A[mid] == key: return comps
        if key < A[mid]: hi = mid - 1
        else: lo = mid + 1
    return comps
```

Listing B.1 – operation counters for M1, M2 and binary search.

```
def T_cur(n): return 1 if n == 1 else 4 * T_cur(n // 2) + n
def T_new(n): return 1 if n == 1 else 2 * T_new(n // 2) + n
def T_m5(n):
    return 1 if n == 1 else 2 * T_m5(n // 2) + n * int(math.log2(n))
def f(n): return 3 * n * n + 10 * n * math.log2(n) + 50
def lg(n): return int(math.log2(n))
def ceil_lg(n): return math.ceil(math.log2(n))
```

Listing B.2 – recurrences for M4 and M5 and the cost function f(n).

```
S = range(2, 3000)
print("M1  n(n-1)/2   :", all(m1_count(n) == n*(n-1)//2 for n in [2, 3, 10, 50, 200]))
print("M2  n=1024     :", m2_count(1024), "  n=1025:", m2_count(1025))
print("M2  formula    :", all(m2_count(n)[0] == 2**ceil_lg(n) - 1 for n in S))
print("M2  bounds     :", all(n-1 <= m2_count(n)[0] <= 2*n-3 for n in S))
for n in [1000, 1023, 1024]:
    A = list(range(0, 2 * n, 2))
    worst = max(bsearch(A, k) for k in range(-1, 2 * n + 1))
    print(f"S2  n={n:<5}     : worst = {worst}, formula = {lg(n) + 1}")
n = 10**6
A = list(range(0, 2 * n, 2))
w = max(bsearch(A, random.randrange(-1, 2 * n + 1)) for _ in range(200000))
print("S2  n=1e6      : worst in 200,000 random keys =", w, ", formula =", lg(n)+1)
K = range(0, 12)
print("M4  current    :", all(T_cur(2**k) == 2 * 4**k - 2**k for k in K))
print("M4  redesign   :", all(T_new(2**k) == k * 2**k + 2**k for k in K))
print("M5  formula    :", all(T_m5(2**k) == 2**k + 2**k*k*(k+1)//2 for k in K))
p, n = 0.5, 4
avg = sum(i * p / n for i in range(1, n + 1)) + n * (1 - p)
print("S1  n=4, p=.5  :", avg, ", formula =", n * (1 - p/2) + p/2)
n = 10**6
print("S1  n=1e6, p=.5: formula =", n * (1 - p/2) + p/2)
ok = all(3*m*m <= f(m) <= 63*m*m for m in range(1, 100001))
print("f   3n^2 <= f(n) <= 63n^2, 1 <= n <= 100000 :", ok)
first = next(m for m in range(2, 1000)
             if all(f(x) <= 4*x*x for x in range(m, 5000)))
print("f   f(n) <= 4n^2 holds for every n >=", first)
```

Listing B.3 – the checks that produce the output below.

```
M1  n(n-1)/2   : True
M2  n=1024     : (1023, 10)   n=1025: (2047, 11)
M2  formula    : True
M2  bounds     : True
S2  n=1000      : worst = 10, formula = 10
S2  n=1023      : worst = 10, formula = 10
S2  n=1024      : worst = 11, formula = 11
S2  n=1e6      : worst in 200,000 random keys = 20 , formula = 20
M4  current    : True
M4  redesign   : True
M5  formula    : True
S1  n=4, p=.5  : 3.25 , formula = 3.25
S1  n=1e6, p=.5: formula = 750000.25
f   3n^2 <= f(n) <= 63n^2, 1 <= n <= 100000 : True
f   f(n) <= 4n^2 holds for every n >= 60
```

Output of the verification program: every check is True and the numerical results agree with the formulas in Parts (b) to (f).

The output confirms: the M1 count equals $n(n-1)/2$; M2 gives 1,023 audits in 10 passes for $n = 1{,}024$ and 2,047 in 11 passes for $n = 1{,}025$; the Audit count satisfies $n - 1 \le A(n) \le 2n - 3$ for all $2 \le n < 3{,}000$; the worst case of binary search equals $\lfloor \log_2 n \rfloor + 1$; the exact solutions of M4 and M5 are correct for $n = 2^0$ to 2^{11}; the S1 average is 3.25 for $n = 4$ and 750,000.25 for $n = 10^6$; and $3n^2 \le f(n) \le 63n^2$ holds for every n from 1 to 100,000, with $f(n) \le 4n^2$ from $n = 60$ onwards.

Appendix C – Extended Growth Tables

The following tables extend the comparison beyond the two catalogue sizes of the scenario. They help to see how each module behaves if the platform keeps growing.

Table 40. Exact operation counts for n from 10^3 to 10^7.

n	1,000	10,000	100,000	1,000,000	10,000,000
M1 $n(n-1)/2$	499,500	5.00×10^7	5.00×10^9	5.00×10^{11}	5.00×10^{13}
M2 $2^{\lceil \log n \rceil} - 1$	1,023	16,383	131,071	1,048,575	1.68×10^7
S2 $\lfloor \log n \rfloor + 1$	10	14	17	20	24
M4 current $2n^2 - n$	1,999,000	2.00×10^8	2.00×10^{10}	2.00×10^{12}	2.00×10^{14}
M4 redesign $n \log n + n$	10,966	142,877	1,760,964	2.09×10^7	2.43×10^8
M5 $n + \tfrac{1}{2} n \log n (\log n + 1)$	55,641	959,255	1.47×10^7	2.10×10^8	2.83×10^9
Report $3n^2 + 10n \log n + 50$	3,099,708	3.01×10^8	3.00×10^{10}	3.00×10^{12}	3.00×10^{14}

Table 41. Running time at 10 operations per second.

n	1,000	10,000	100,000	1,000,000	10,000,000
M1	13.9 hours	57.9 days	16 years	1,584 years	158,440 years
M2	102.3 s	27.3 min	3.6 hours	29.1 hours	19.4 days
S2	1.0 s	1.4 s	1.7 s	2.0 s	2.4 s
M4 current	2.3 days	231.5 days	63 years	6,338 years	633,762 years
M4 redesign	18.3 min	4.0 hours	2.0 days	24.2 days	280.7 days
M5	92.7 min	26.6 hours	17.0 days	242.6 days	9 years
Report	3.6 days	348.8 days	95 years	9,507 years	950,650 years

Table 42. Growth factor for each tenfold increase in n.

From → to	M1	M2	S2	M4 current	M4 redesign	M5
1,000 → 10,000	× 100.09	× 16.01	× 1.40	× 100.05	× 13.03	× 17.24
10,000 → 100,000	× 100.01	× 8.00	× 1.21	× 100.00	× 12.33	× 15.35
100,000 → 1,000,000	× 100.00	× 8.00	× 1.18	× 100.00	× 11.89	× 14.23
1,000,000 → 10,000,000	× 100.00	× 16.00	× 1.20	× 100.00	× 11.59	× 13.50

Each tenfold increase multiplies the quadratic modules by about 100, M5 by about 13.5 to 17, the redesigned M4 by about 11.6 to 13, and M2 by 8 or 16 (depending on how many powers of two the increase crosses). S2 grows by a factor of at most 1.4 in the range shown, because it gains only three or four comparisons.

Appendix D – Notation and References

Notation

Table 43. Notation used in the report.

Symbol	Meaning
n	input size (products, requests or records)
T(n)	number of basic operations of a recursive algorithm
a, b, f(n)	number of sub-problems, shrink factor and divide/merge cost in T(n) = aT(n/b) + f(n)
log n	logarithm to base 2 unless another base is shown
⌊x⌋, ⌈x⌉	floor and ceiling of x
O, Ω, Θ	asymptotic upper, lower and tight bounds
p	probability that the searched product exists in the catalogue
K	number of passes of the WHILE loop in M2, K = $\lceil \log_2 n \rceil$
A(n)	number of Audit calls made by M2
C_{avg}	average number of comparisons

References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022 – asymptotic notation, recurrences and the Master Theorem.
2. A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Pearson, 2012 – basic operations, mathematical analysis of recursive and non-recursive algorithms.
3. R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011 – binary search, sorting and order-of-growth classification.
4. D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998 – analysis of sequential and binary search.
5. M. Akra and L. Bazzi, “On the solution of linear recurrence equations”, *Computational Optimization and Applications*, 10(2), 1998 – generalisation of the Master Theorem.